

AUDITSYNC

AI-Powered Revenue Reconciliation Platform

Product & Engineering Case Study

Prepared by Ayesha Jabeen Khan

AI-Integrated Systems | Automation Engineering

Executive Summary

Companies running separate CRM, payment gateway, and billing systems routinely lose 5–10% of revenue to reconciliation gaps — duplicate charges, invoices that never get generated, and failed payments that are never retried. These gaps are typically caught, if at all, during a manual month-end review, by which point the money is often uncollectable.

AuditSync is a full-stack platform I designed and built to close that gap automatically. It reconciles transactions across all three data sources, uses AI to explain the root cause of every discrepancy in plain English, and alerts the finance team — turning a manual spreadsheet exercise into an automatic safety net. Visitors can run the tool against a seeded demo dataset, or upload their own payment and billing data and watch it reconcile in real time, right from the live demo.

Live deployment: audit-sync.vercel.app · API: auditsync.onrender.com

Frontend hosted on Vercel, backend API on Render, database on Neon (serverless PostgreSQL). The API may take a few seconds to respond on first request after idling. The live demo sits behind a simple password gate — ask for access.

TOTAL AT RISK (SAMPLE RUN)	OPEN LEAKS FLAGGED	LEAK TYPES DETECTED
\$6,012.66	18	4

The Problem

Finance and operations teams reconciling revenue across disconnected systems face four recurring, expensive failure modes:

- Missing in billing — a payment succeeds but is never invoiced.
 - Duplicate charges — the same customer is billed twice for the same amount within 24 hours.
 - Amount mismatches — the payment processor and billing system disagree on what was charged.
 - Failed payments, never retried — a charge fails and silently falls through the cracks.
- Caught manually, these issues surface weeks or months late. AuditSync catches them the next morning.

System Architecture

The platform is built as four cooperating layers — automation, backend reconciliation, AI explanation, and a reporting frontend — so each part can scale or be replaced independently.

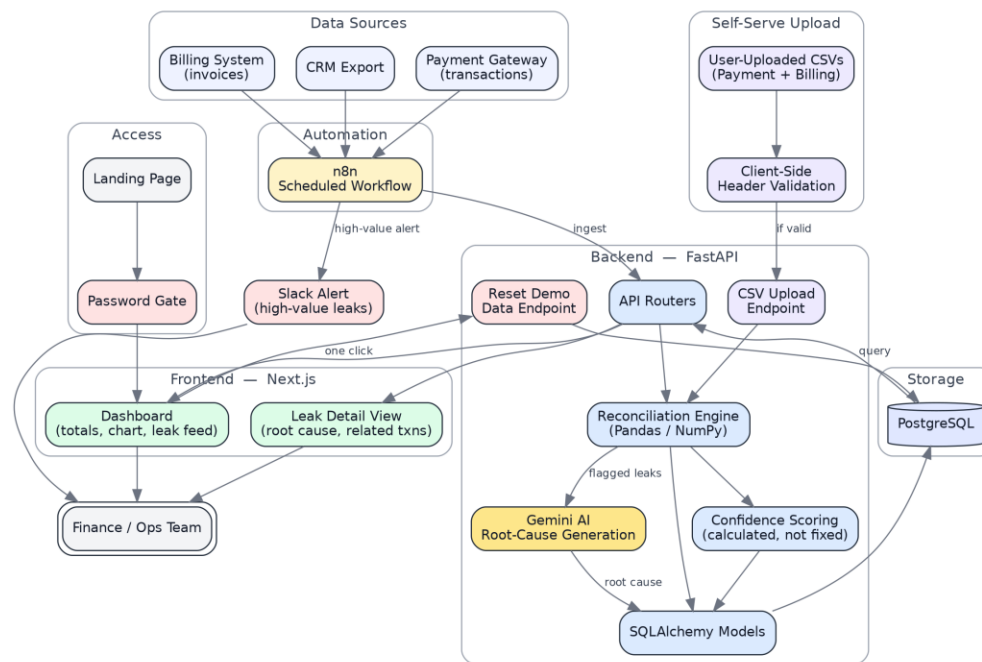


Figure 1 — End-to-end data flow: access gate, ingestion (scheduled or direct CSV upload), reconciliation and confidence scoring, AI analysis, and reporting.

How it flows

- A visitor lands on the marketing page, then passes through a lightweight password gate before reaching the actual tool.
- Data arrives either through a scheduled workflow pulling from the CRM, payment gateway, and billing system, or a user uploads a Payment Processor CSV and Billing CSV directly — validated for the right columns in the browser before anything is sent.
- FastAPI ingests the data through the same reconciliation path either way; a Pandas/NumPy engine cross-checks every transaction across sources.
- Each discrepancy goes through a confidence-scoring step — calculated from the actual match signal, not a fixed number — then to Gemini AI, which generates a plain-English root-cause explanation.
- Results are persisted in PostgreSQL and served through a REST API; a one-click reset endpoint restores the clean demo state.
- High-value leaks trigger an immediate Slack alert; everything else surfaces on the dashboard.

Product Walkthrough

1. Landing page

A clear framing of the problem, real stats from processed runs, and a one-click way to try the tool.

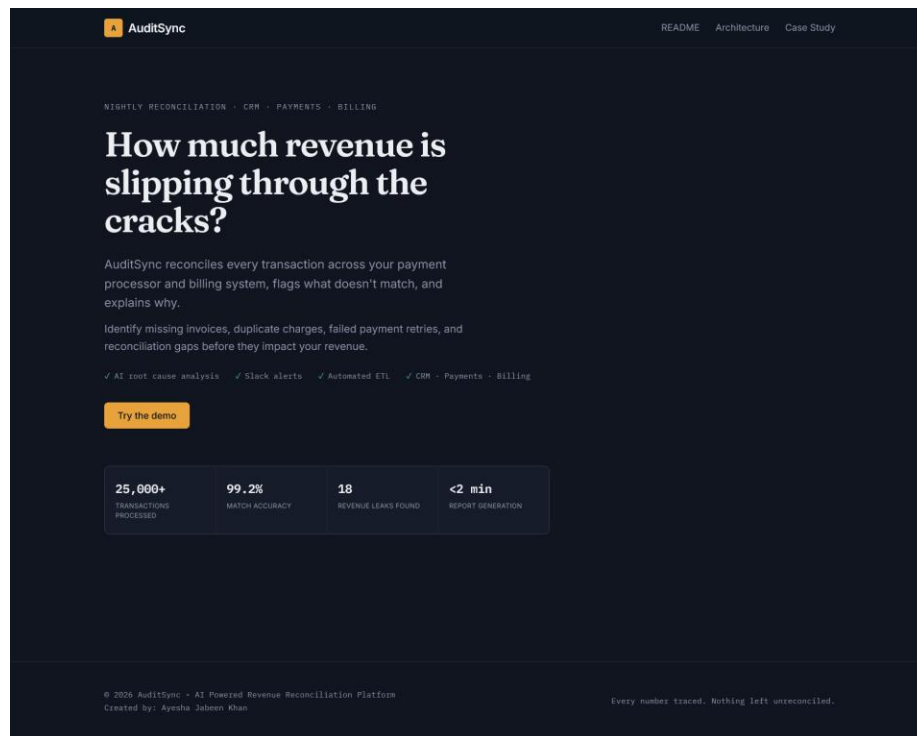


Figure 2 — Landing page: value proposition, live stats, and the entry point into the demo.

2. The access gate

Since the live demo is a single public link, a lightweight password screen sits in front of the actual tool — enough to keep it from becoming an open sandbox, without needing full user accounts for what's still a portfolio demo.

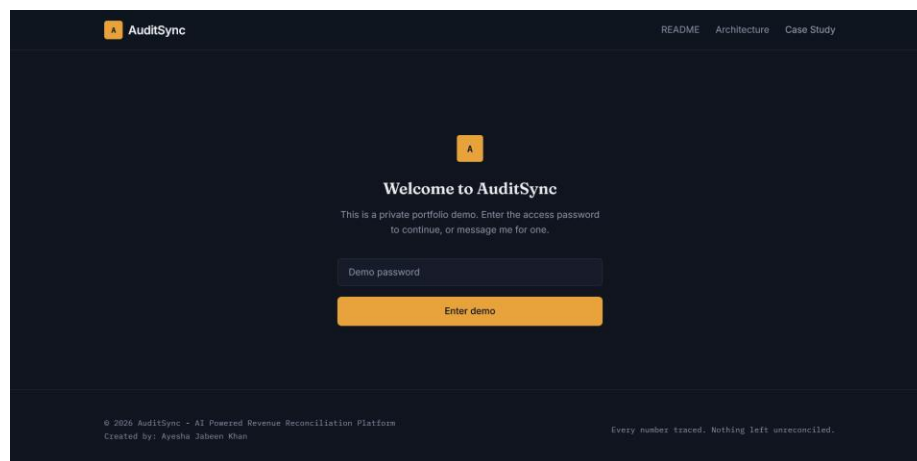


Figure 3 — Password-gated entry point to the working dashboard.

3. Bring your own data

A self-serve panel lets a visitor upload their own Payment Processor CSV and Billing CSV and run the exact same reconciliation pipeline against it — not a simplified preview. Column headers are validated in the browser before anything is sent to the server, and the panel shows exactly how many rows were ingested versus rejected. Sample CSVs are one click away for anyone who wants to see the expected format first.

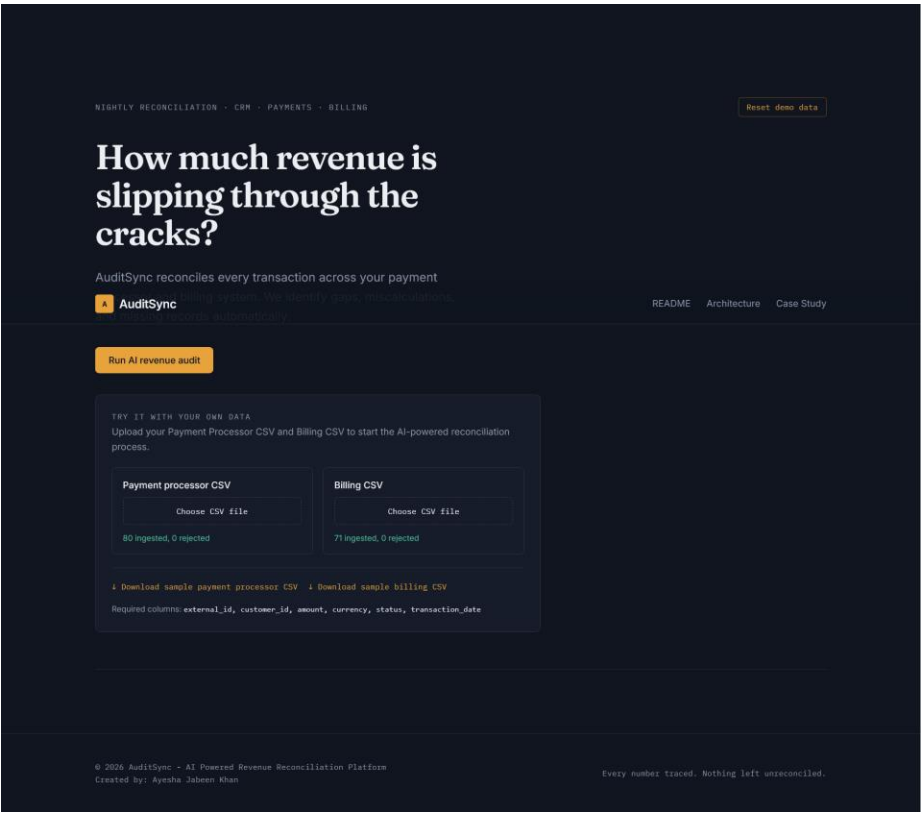


Figure 4 — CSV upload panel with client-side validation and live ingestion counts.

4. Dashboard overview

Once data is in, the dashboard shows total revenue at risk, how many leaks are open, and how many distinct leak types have been detected — plus a time-series view of leakage.

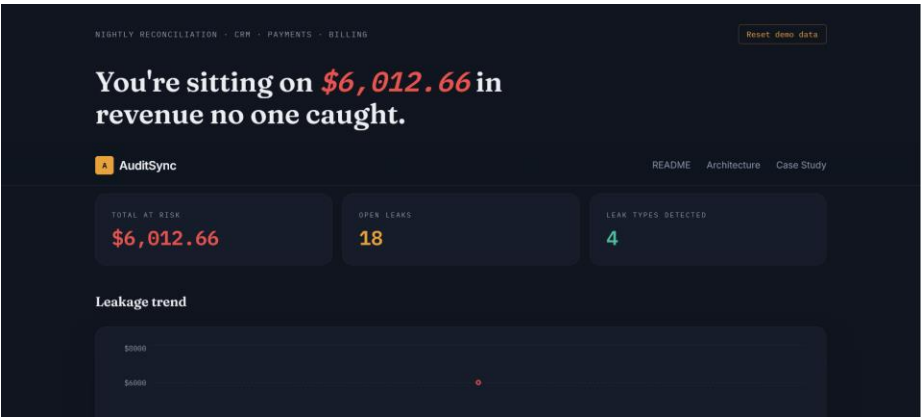


Figure 5 — Dashboard summary: total at risk, open leak count, and leak types detected, trended over time.

5. Flagged leaks feed

Every leak is listed with its type, affected customer, a confidence score, and dollar amount at risk — ranked so the highest-confidence, highest-value issues surface first. Each confidence score is calculated from the underlying transaction signal, not a fixed number per leak type.

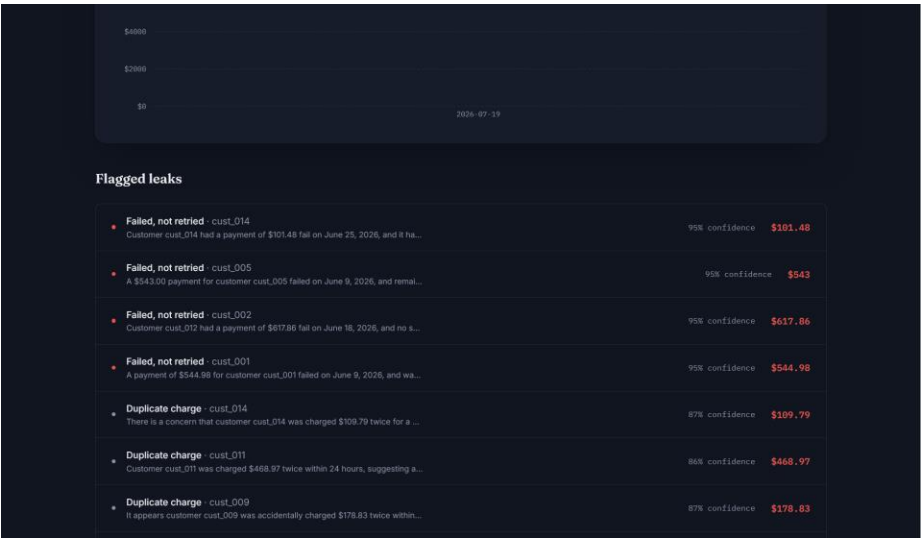


Figure 6 — Flagged leaks feed, spanning failed retries, duplicate charges, amount mismatches, and missing invoices.

6. Leak detail & root cause

Clicking into a leak shows the full AI-generated root-cause explanation, the exact related transaction IDs, and a status the finance team can update as they resolve it.

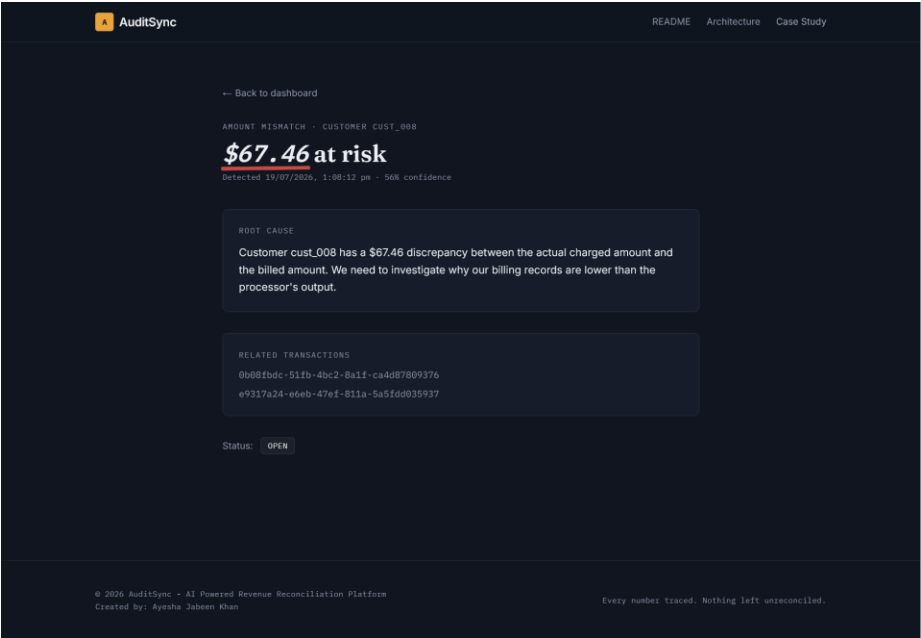


Figure 7 — Leak detail view: an amount mismatch, root cause, related transaction IDs, and open status.

What AuditSync Detects

Leak Type	What it means	Signal
Missing in billing	Payment completed, but the transaction was never invoiced.	High
Duplicate charge	Same customer charged the same amount twice within 24 hours.	High

Amount mismatch	Payment processor and billing system disagree on the charged amount.	Medium
Failed, not retried	A failed payment that was never retried — silent, permanent loss.	High

Technology Stack

Frontend

Next.js (App Router), TypeScript, TailwindCSS, shadcn/ui, Recharts — fully mobile-responsive.

Backend

FastAPI, Pydantic, SQLAlchemy, Pandas / NumPy for the reconciliation engine.

AI & Automation

Gemini API for root-cause generation, batched per reconciliation run to control cost and stay within rate limits. n8n orchestrates the nightly workflow and Slack alerting.

Infrastructure

Neon (serverless PostgreSQL) for the database, Docker & docker-compose for local development, deployed live on Vercel (frontend) and Render (backend API).

Engineering Highlights

A few of the real problems solved during development — the kind of detail that matters once a system is actually being used, not just demoed once:

An idempotency bug, found by using my own tool

Clicking “reconcile” twice in the running app was silently duplicating every leak — not caught in initial testing, found by using the product the way a real user would. Fixed with a per-transaction “reconciled” watermark, so re-running the job is always safe no matter how many times it's triggered.

Confidence scores that were quietly fake

An earlier version returned a flat confidence number per leak type — always 90% for one category, always 85% for another. It looked correct in a demo but wasn't measuring anything. Fixed so confidence is now calculated from the real signal behind each finding: how large a missing payment is, how close together duplicate charges are, how long a failed payment has gone unretried. Hardcoded numbers are an easy mistake precisely because they never look obviously wrong.

Client-side validation before the network call

CSV column headers are checked in the browser before the file is sent anywhere. A malformed upload fails instantly with a specific message instead of round-tripping to the server first — a small detail that changes whether a demo feels like a real product or a fragile prototype.

A password gate sized to the actual problem

The demo needed to be a single link I could post publicly without it turning into an open sandbox, but didn't need full user accounts at this stage. A lightweight password screen was the right-sized solution — not under-engineered, not over-built for a problem that doesn't exist yet.

Accurate failed-payment matching

A completed transaction for an unrelated purchase was initially misread as resolving an earlier failed payment. Fixed by requiring a retry to match the original amount before marking it resolved, with a regression test added to lock in the fix.

Resilient AI integration

Root-cause calls are batched per run rather than per leak to respect API quotas, with an automatic templated fallback if Gemini is rate-limited or briefly unavailable — so a third-party outage never breaks a live run.

About This Project

AuditSync was designed and built end-to-end — data modeling, reconciliation logic, AI integration, automation, and the reporting interface — as a demonstration of how a small, well-architected system can replace a recurring manual finance process.

Ayesha Jabeen Khan

AI-Integrated Systems & Automation